

Elaborate various measures of Central Tendencies with respect to data analysis with suitable examples.

Measures of central tendency

Mean

The **mean** (or average) is the value obtained by dividing the sum of all observations by the total number of observations.

Direct method	Assumed mean method	Step deviation method
Mean $\bar{x} = \frac{\sum_{i=1}^n f_i x_i}{\sum_{i=1}^n f_i}$ Here, $\sum f_i$ = Sum of all frequencies	Mean $(\bar{x}) = a + \frac{\sum f_i d_i}{\sum f_i}$ Here, a = Assumed mean $d_i = x_i - a$ $\sum f_i$ = Sum of all frequencies	Mean $(\bar{x}) = a + h \frac{\sum f_i u_i}{\sum f_i}$ Here, a = Assumed mean $u_i = (x_i - a)/h$ h = Class size $\sum f_i$ = Sum of all frequencies

Find the mean of the following data.

Class Interval	0-10	10-20	20-30	30-40	40-50
Frequency	12	16	6	7	9

Solution:

We may prepare the table given below:

Class Interval	Frequency f_i	Class Mark x_i	($f_i \cdot x_i$)
0-10	12	5	60
10-20	16	15	240

20-30	6	25	150
30-40	7	35	245
40-50	9	45	405
	$\sum f_i = 50$		$\sum f_i \cdot x_i = 1100$

$$\text{Mean} = \frac{\sum (f_i \cdot x_i)}{\sum f_i} = \frac{1100}{50} = 22$$

Median

Generally median represents the mid-value of the given set of data when arranged in a particular order.

Median: Given that the data collection is arranged in ascending or descending order, the following method is applied:

- If number of values or observations in the given data is odd, then the median is given by $[(n+1)/2]^{\text{th}}$ observation.
- If in the given data set, the number of values or observations is even, then the median is given by the average of $(n/2)^{\text{th}}$ and $[(n/2) + 1]^{\text{th}}$ observation.

The median for grouped data can be calculated using the formula

$$\text{Median} = l + \left(\frac{\frac{N}{2} - cf}{f} \right) \times h$$

Example: Calculate the median for the following frequency distribution.

Class Interval	0-8	8-16	16-24	24-32	32-40	40-48
Frequency	8	10	16	24	15	7

Solution:

We may prepare cumulative frequency table as given below,

Class	Frequency	Cumulative Frequency
0-8	8	8
8-16	10	18
16-24	16	34
24-32	24	58
32-40	15	73
40-48	7	80
	$N = \sum f_i = 80$	

Now, $N = 80 = (N/2) = 40$.

The cumulative frequency just greater than 40 is 58 and the corresponding class is 24-32.

Thus, the median class is 24-32.

$l = 24$, $h = 8$, $f = 24$, $c_f = \text{c.f. of preceding class} = 34$, and $(N/2) = 40$.

$$\text{Median} = l + \left(\frac{\frac{N}{2} - c_f}{f} \right) \times h$$

...

$$= 24 + 8 \left\{ \frac{(40 - 34)}{24} \right\}$$

$$= 26$$

Hence, **median = 26**.

Mode

The most frequent number occurring in the data set is known as the mode.

Consider the following data set which represents the marks obtained by different students in a subject.

Name	Anmol	Kushagra	Garima	Ashwini	Geetika	Shakshi
Marks Obtained (out of 100)	73	80	73	70	73	65

The maximum frequency observation is 73 (as three students scored 73 marks), so the mode of the given data collection is 73.

We can calculate the mode for grouped data using the below formula:

$$\text{Mode} = l + \left(\frac{f_1 - f_0}{2f_1 - f_0 - f_2} \right) \times h$$

A survey has been conducted by a group of students on 20 households in a locality as shown in the following frequency distribution table. Find the mode for the given data.

Size of Family	1-3	3-5	5-7	7-9	9-11
No. of Families	7	8	2	2	1

Solution:

From the given table, it is observed that the maximum class frequency is 8, and the corresponding class interval is 3-5.

Therefore, the modal class for the given data is 3-5.

The lower limit of modal class, $l = 3$

Class size, $h = 2$

Frequency of modal class, $f_1 = 8$

Frequency of class proceeding to modal class, $f_0 = 7$

Frequency of class succeeding to modal class, $f_2 = 2$

We know that the formula to find the mode of the grouped data is:

$$\text{Mode} = l + \left(\frac{f_1 - f_0}{2f_1 - f_0 - f_2} \right) \times h$$

Now, substituting the values in the [mode formula](#), we get,

$$\text{Mode} = 3 + \left(\frac{8-7}{2(8)-7-2} \right) \times 2$$

$$\text{Mode} = 3 + (2/7)$$

$$\text{Mode} = (21+2)/7$$

$$\text{Mode} = 23/7$$

$$\text{Mode} = 3.286.$$

Therefore, the mode of the given grouped data is 3.286.

Relation of Mean Median Mode

$$\text{Mode} = 3 \text{ Median} - 2 \text{ Mean}$$

Comparison

Measure	Advantages	Disadvantages	Uses
Mean	• Uses all data values. • Good for further statistical analysis (variance, standard deviation).	• Strongly affected by extreme values (outliers) • Not suitable for skewed data.	• Commonly used in economics (average income, GDP) • In education (average marks) • In science and research (average results of experiments).

		• May not represent actual data if distribution is irregular.	
Median	• Not affected by extreme values or outliers. • Represents the central tendency well in skewed distributions.	• Does not use all observations. • Calculation can be tedious for large ungrouped datasets. • Less suitable for further statistical analysis.	• Used in income/wealth studies (median income). • In real estate (median house price). • In medical data (median survival time).
Mode	• Can be used for categorical data (like most common color, brand, etc.). • Not affected by extreme values.	• May not exist (if all values occur equally). • May not be unique (more than one mode). • Not based on all data values.	• Used in market research (most preferred product). • In quality control (most common defect type). • In demographics (most common age group, shoe size, etc.).

Demonstrate different data visualization techniques

Definition:

- The graphical representation of information and data is known as data visualization.
- These visualizations help users draw insights.
- They provide an easy way to understand datasets in terms of data patterns, such as:
Trends Presence of outliers

Basic Charts

1. Bar Chart

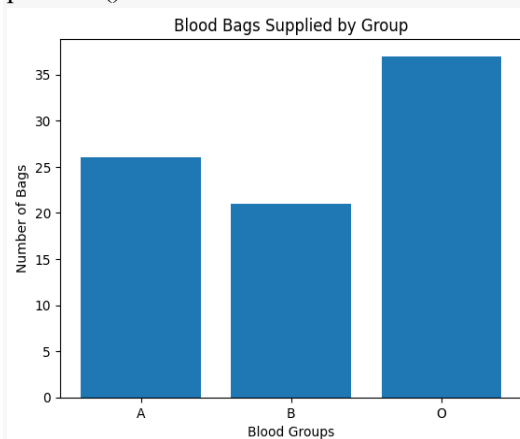
It is a graphical representation of data attributes using rectangular bars with heights or lengths proportional to the values they represent.

The number of plastic bags supplied for each group is shown in the table below:

Blood Group	bags
A	26
B	21
C	37

```
import matplotlib.pyplot as plt
```

```
blood_groups = ['A', 'B', 'O']  
bags = [26, 21, 37]  
plt.bar(blood_groups, bags)  
plt.xlabel("Blood Groups")  
plt.ylabel("Number of Bags")  
plt.title("Blood Bags Supplied by Group")  
plt.show()
```



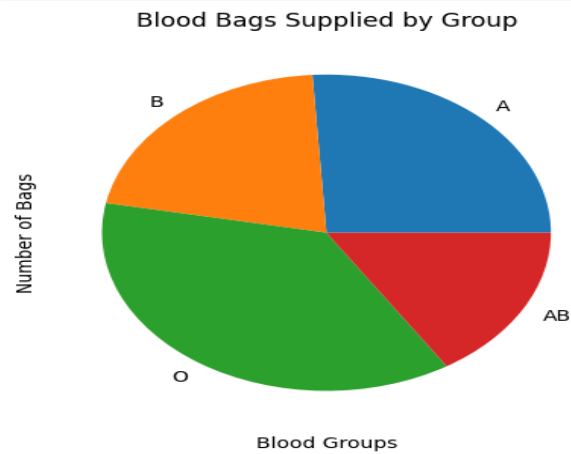
Pie Chart

It consists of a circle with several sectors, one for each attribute

The chart's fraction is the sum of each category in an attribute

```
import matplotlib.pyplot as plt  
blood_groups = ['A', 'B', 'O', 'AB']  
bags = [26, 21, 37, 16]  
plt.pie(bags, labels=blood_groups)  
plt.xlabel("Blood Groups")  
plt.ylabel("Number of Bags")
```

```
plt.title("Blood Bags Supplied by Group")
plt.show()
```



Histogram

- Shows frequency distribution of continuous data.
- They visually display data points organized into specified ranges determined by the user.

Class Interval	Frequency
0-10	5
11-20	12

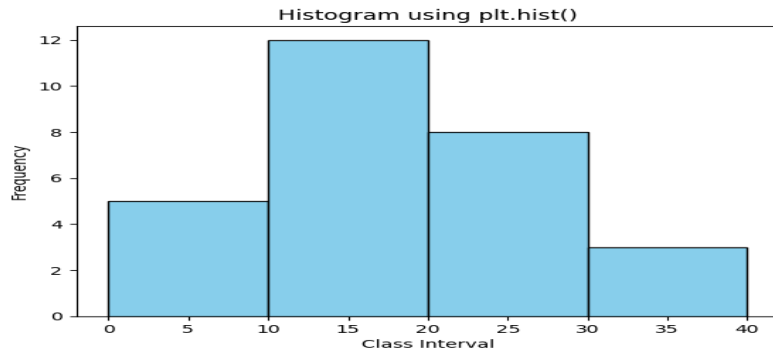
```
import matplotlib.pyplot as plt

# Data
bins = [0, 10, 20, 30, 40] # class intervals
frequencies = [5, 12, 8, 3] # given frequencies

# Use the midpoints as data, and assign weights
midpoints = [5, 15, 25, 35]

plt.hist(midpoints, bins=bins, weights=frequencies,
         color='skyblue', edgecolor='black')

plt.xlabel("Class Interval")
plt.ylabel("Frequency")
plt.title("Histogram using plt.hist()")
plt.show()
```

Box Plot

A box plot, also known as a box-and-whisker plot, is a standardized way of displaying the distribution of data based on a five-number summary:

Here are the runs scored by a cricket team in a league of 12 matches - **100, 120, 110, 150, 110, 140, 130, 170, 120, 220, 140, 110.**

Ascending Order

100, 110, 110, 110, 120, 120, 130, 140, 140, 150, 170, 220

Elements of Box Plot

A box plot gives a five-number summary of a set of data which is-

- **Median (Q2)** - It is the mid-point of the dataset. Half of the values lie below it and half above.

$$\text{Median (Q2)} = (120+130)/2 = \mathbf{125};$$

- **First Quartile (Q1)** - 25% of the data lies below the First (lower) Quartile.

$$\text{Q1} = (110+110)/2 = \mathbf{110}$$

- **Third Quartile (Q3)** - 75% of the data lies below the Third (Upper) Quartile.

$$\text{Q3} = (140+150)/2 = \mathbf{145}$$

$$\text{IQR} = \text{Q3} - \text{Q1} = 145 - 110 = \mathbf{35}$$

$$\text{Lower Limit} = \text{Q1} - 1.5 * \text{IQR} = 110 - 1.5 * 35 = \mathbf{57.5}$$

$$\text{Upper Limit} = \text{Q3} + 1.5 * \text{IQR} = 145 + 1.5 * 35 = \mathbf{197.5}$$

- **Minimum** - It is the minimum value in the dataset excluding the outliers.

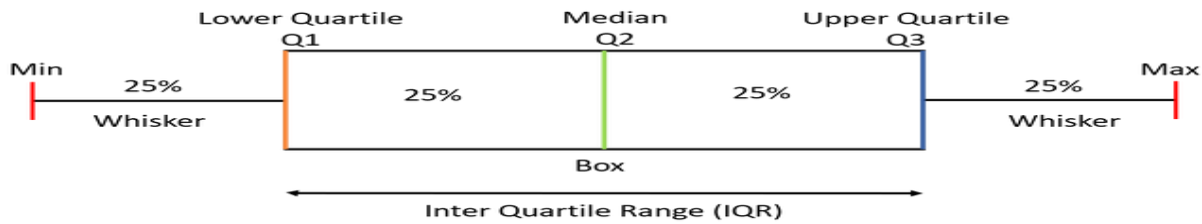
- **Maximum** - It is the maximum value in the dataset excluding the outliers.

So, the minimum and maximum between the range [57.5, 197.5] for our given data are -

$$\text{Minimum} = \mathbf{100}$$

$$\text{Maximum} = \mathbf{170}$$

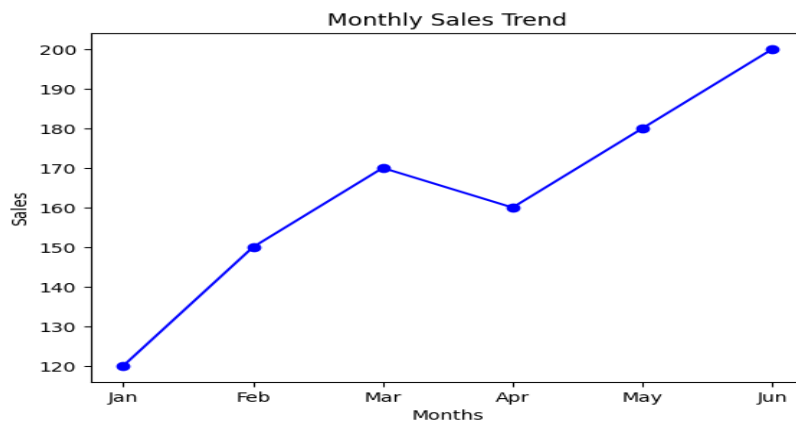
$$\text{Outliers} = \mathbf{220}$$



Line chart

A line chart is a type of chart that displays a series of data points connected by straight line segments to show how data changes over continuous intervals, often time, against a common scale

```
import matplotlib.pyplot as plt
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [120, 150, 170, 160, 180, 200]
plt.plot(months, sales, marker='o', color='blue', linestyle='-')
plt.xlabel("Months")
plt.ylabel("Sales")
plt.title("Monthly Sales Trend")
plt.show()
```



Advanced Charts

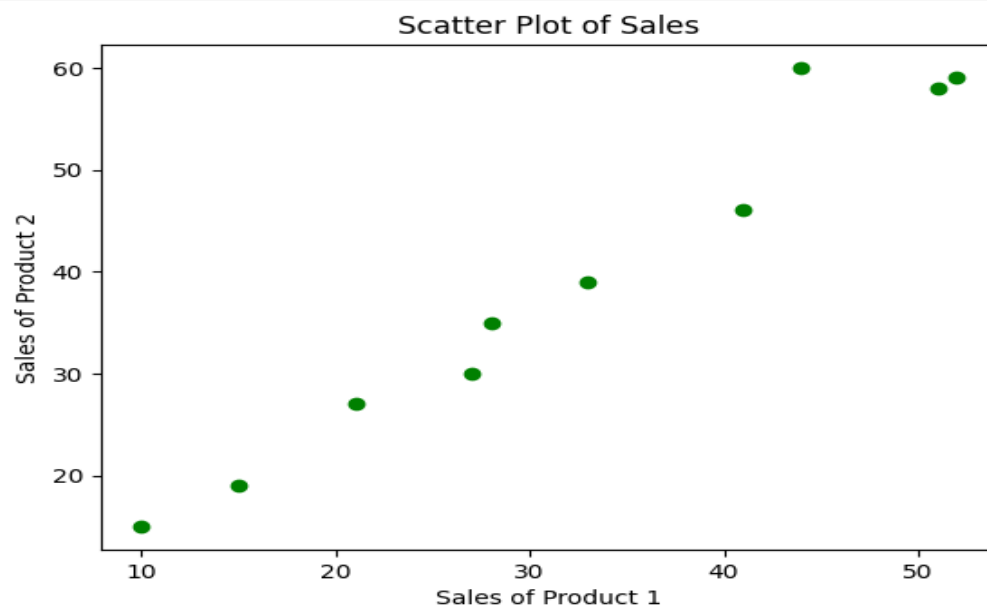
5. Scatter Plot

- These graphs display the relationship between two variables in a dataset.
- It can be determined by observing whether the data points are scattered across the graph or if they form a band between two Variables
- Scattered data indicates that the variables are unrelated.
- A band indicates that the variables are related.

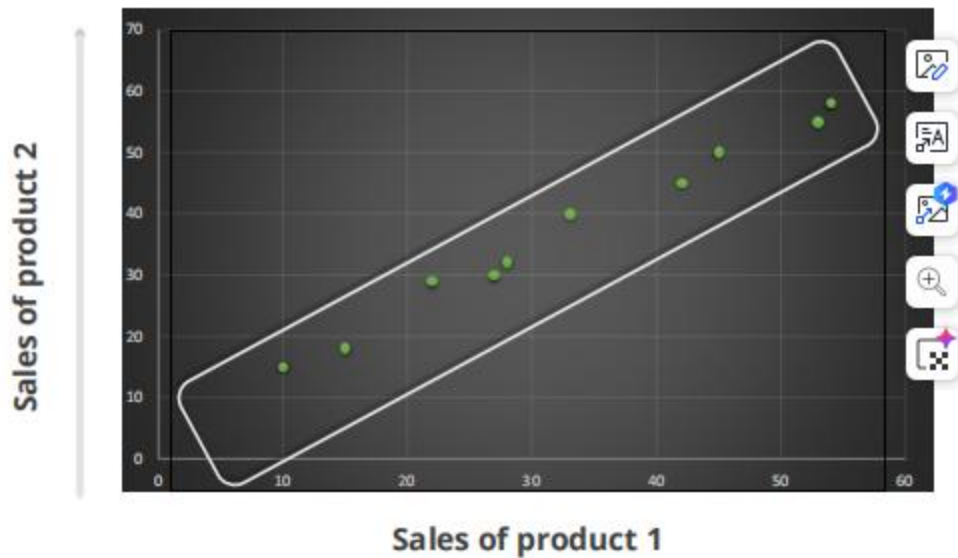
The following data represents the sales of two products at a retail outlet over a 10-day period

Product1	10	15	21	27	28	33	41	44	51	52
Product 2	15	19	27	30	35	39	46	60	58	59

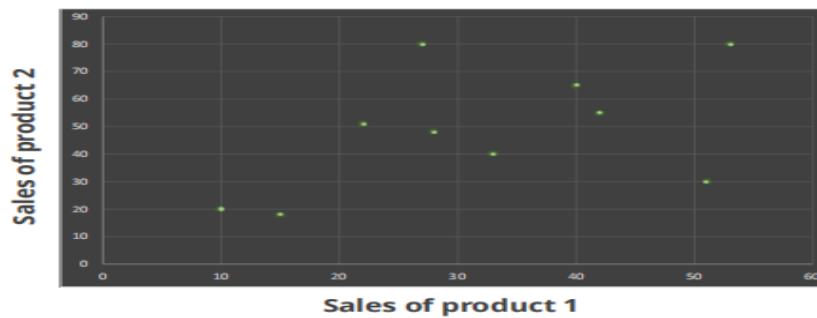
```
import matplotlib.pyplot as plt
product1 = [10, 15, 21, 27, 28, 33, 41, 44, 51, 52]
product2 = [15, 19, 27, 30, 35, 39, 46, 60, 58, 59]
plt.scatter(product1, product2, color='green')
plt.xlabel("Sales of Product 1")
plt.ylabel("Sales of Product 2")
plt.title("Scatter Plot of Sales")
plt.show()
```



A narrow band indicates a relationship between the two variables, such as the sales of two products on different days.



Plotted values are scattered across the chart and values do not fall in the band



In such cases, there is no relationship between the variables

Bubble Plot

This is an extension of the scatter plot used to identify relationships between three numerical variables.

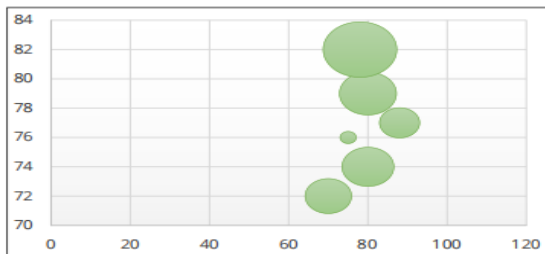
It is a data visualization that uses bubbles to represent data points, with the size of the bubble indicating a third dimension of data.

Example: Plot a bubble chart using the three variables in the given data

Variable 1	78	80	88	78	70	75
Variable 2	82	79	77	74	72	76
Variable 3	87	79	77	80	78	74

```
import matplotlib.pyplot as plt
x = [78, 80, 88, 78, 70, 75]
y = [82, 79, 77, 74, 72, 76]
sizes = [87, 79, 77, 80, 78, 74] # third variable
plt.scatter(x, y, s=sizes, alpha=0.5, c='purple')
plt.xlabel("Variable 1")
plt.ylabel("Variable 2")
plt.title("Bubble Plot Example")
plt.show()
```

- Extension of scatter plot (third variable = bubble size).



- The first variable varies over a broader range, and hence the Y-axis is spread over a wider range.
- The reduced spread of the second variable results in a narrower range on the X - axis.

Chart Type	Data Type Used	Purpose	Uses
Bar Chart	Categorical / Qualitative	Compare categories	Useful when comparing discrete groups
Pie Chart	Categorical (Proportions)	Show part-to-whole relationships	Best when total = 100% and categories are few
Histogram	Quantitative (Continuous)	Show frequency distribution	Useful for understanding shape of distribution
Box Plot	Quantitative	Summarize spread & detect outliers	Best for comparing data spread & identifying outliers

Scatter Plot	Quantitative (2 variables)	Show relationship between two variables	Useful for correlation & trend analysis
Bubble Plot	Quantitative (3 variables)	Show relationship between three variables	Good for multi-dimensional analysis
Line Chart	Quantitative (Time series or ordered data)	Show trends over time or continuous changes	Useful for trend analysis & forecasting

Exemplify Data Cleaning & Data Munging Techniques

Data Cleaning

- Process of **detecting and correcting errors** in datasets.
- Ensures data is accurate, consistent, and usable.
- Essential before analysis, machine learning, or visualization.

Data Munging (Wrangling)

- Process of **transforming raw data into a structured format** suitable for analysis.
- Includes **cleaning, reshaping, merging, aggregating, and enriching data**.
- Goal: Make raw messy data → **tidy, structured, analysis-ready** data.

Techniques

1. Handling Missing Data

- **Problem:** Missing values (NaN) can cause errors in analysis.
- **Techniques:**
 - Remove missing rows/columns.
 - Fill with mean/median/mode.
 - Forward/Backward fill for time series.

```
import pandas as pd
import numpy as np
data = {'Age': [25, np.nan, 30, 28, np.nan],
        'Salary': [50000, 54000, np.nan, 58000, 60000]}
df = pd.DataFrame(data)
```

```

print("DATA WITH MISSING VALUE\n")
print(df)
# Fill missing Age with mean
df['Age'] = df['Age'].fillna(df['Age'].mean())
# Drop rows with missing Salary
df = df.dropna(subset=['Salary'])
print("DATA AFTER FILLING AND REMOVING MISSING VALUE\n")
print(df)

```

DATA WITH MISSING VALUE

	Age	Salary
0	25.0	50000.0
1	NaN	54000.0
2	30.0	NaN
3	28.0	58000.0
4	NaN	60000.0

DATA AFTER FILLING AND REMOVING MISSING VALUE

	Age	Salary
0	25.000000	50000.0
1	27.666667	54000.0
3	28.000000	58000.0
4	27.666667	60000.0

2. Removing Duplicates

- **Problem:** Duplicated rows distort statistics.
- **Solution:** Use `drop_duplicates()`.

```

data = {'Name': ['Alice', 'Bob', 'Alice'],
        'Age': [25, 30, 25]}
df = pd.DataFrame(data)
df = df.drop_duplicates()
print(df)

```

	Name	Age
0	Alice	25
1	Bob	30

3. Fixing Inconsistent Data

- **Issues:** Wrong spellings, inconsistent capitalization, mixed formats.
- **Fix:** Standardize text, convert cases, format dates.

```
data = {'City': ['New york', 'new York', 'NEW YORK', 'London']}
df = pd.DataFrame(data)
df['City'] = df['City'].str.title() # Standardize to Title Case
print(df)
```

```
      City
0  New York
1  New York
2  New York
3   London
```

4. Outlier Detection & Treatment

- Outliers can skew results.
- Methods:
 - Z-Score method.
 - IQR method (Interquartile Range).
 - Winsorization (replace with boundary values).

Example (IQR method)

```
import numpy as np
data = [10, 12, 15, 14, 13, 100, 12, 11] # 100 = outlier
Q1, Q3 = np.percentile(data, [25, 75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
cleaned = [x for x in data if x >= lower and x <= upper]
print("After Removing Outliers:", cleaned)
```

After Removing Outliers: [10, 12, 15, 14, 13, 12, 11]

5. Data Transformation

- Normalization, Scaling, Encoding categorical variables.

Example – Normalization

```
from sklearn.preprocessing import MinMaxScaler
data = [[100], [200], [300], [400]]
scaler = MinMaxScaler()
normalized = scaler.fit_transform(data)
print(normalized)
```



```
[0.      ]
[0.33333333]
[0.66666667]
[1.      ]]
```

6. Data Integration & Merging

- Combine datasets from multiple sources.

Example

```
df1 = pd.DataFrame({'ID': [1,2,3], 'Name': ['A','B','C']})
df2 = pd.DataFrame({'ID': [1,2,3], 'Salary': [50000,60000,55000]})
merged = pd.merge(df1, df2, on='ID')
print(merged)
```

```
   ID  Name  Salary
0    1    A   50000
1    2    B   60000
2    3    C   55000
```

Compare and contrast named tuple and data class

Feature	namedtuple	dataclass
Definition	Lightweight, immutable tuple subclass with named fields	Class decorator that automatically adds methods like <code>init</code> , <code>repr</code> , <code>eq</code>
Mutability	Immutable (fields cannot be changed after creation)	Mutable by default (but can be frozen to make immutable)
Memory Efficiency	More memory-efficient (tuple-based)	Slightly heavier (class-based)
Default Values	Not supported directly (need <code>_replace()</code> or defaults via defaults)	Supported (<code>field(default=...)</code>)
Methods	Limited methods (no custom methods by default)	Can define custom methods easily
Inheritance	Cannot inherit (tuple subclass)	Supports inheritance
Use Case	Good for lightweight, immutable records Example: coordinates, RGB values, database query results	Good for richer, mutable objects with behaviors

Example:

```
from collections import namedtuple
```

```
# Define Student using namedtuple
Student = namedtuple("Student", ["name", "age", "marks"])

s1 = Student("Anita", 21, 85)

print(s1)      # Student(name='Anita', age=21, marks=85)
print(s1.name) # Access by name → Anita
print(s1[2])   # Access by index → 85

# ✗ Cannot change values (immutable)
# s1.age = 22 # AttributeError
```

Output:

```
Student(name='Anita', age=21, marks=85)
Anita
85
```

```
s1.age = 22
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-3801868558.py in <cell line: 0>()
----> 1 s1.age = 22

AttributeError: can't set attribute
```

Using dataclass

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int
    marks: int = 0 # Default value allowed

s2 = Student("Anita", 21, 85)

print(s2)      # Student(name='Anita', age=21, marks=85)
print(s2.name) # Access → Anita

s2.age = 22    # ✓ Can modify (mutable)
```

```
print(s2)    # Student(name='Anita', age=22, marks=85)
```

OUTPUT:

```
Student(name='Anita', age=21, marks=85)
Anita
Student(name='Anita', age=22, marks=85)
```

Paraphrase different File operations

Reading from a Text File ("r" mode) Writing content to the file

```
# First, create a file
```

```
with open("notes.txt", "w") as f:
    f.write("Line 1: Python File Handling\n")
    f.write("Line 2: Read, Write, Append\n")
```

```
# Read the file
```

```
with open("notes.txt", "r") as f:
    content = f.read()
```

```
print("📄 File Content in Read Mode:")
print(content)
```

OUTPUT:

```
📄 File Content in Read Mode:
Line 1: Python File Handling
Line 2: Read, Write, Append
```

Appending to a Text File ("a" mode)":

```
with open("notes.txt", "a") as f:
    f.write("This line is appended using append mode.\n")
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

OUTPUT:

```
Line 1: Python File Handling
Line 2: Read, Write, Append
This line is appended using append mode.
```

Reading and writing operations with delimited files

A delimited file is just a plain text file with a chosen separator (comma, tab, |, etc.)

Creating and reading from csv file

```
import csv

# Writing data into a CSV file
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    # Writing header
    writer.writerow(["Name", "Age", "Marks"])
    # Writing rows
    writer.writerow(["Anita", 21, 85])
    writer.writerow(["Rahul", 22, 90])
    writer.writerow(["Vidya", 23, 95])
```

```
print(" Data written to students.csv")
```

Output:

Students.csv file

Name,Age,Marks

Anita,21,85

Rahul,22,90

Vidya,23,95

```
print(" Data written to students.csv")
```

Reading from a CSV File:

```
import csv

# Reading data from a CSV file
with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

OUTPUT:

```
[ 'Name', 'Age', 'Marks' ]
[ 'Anita', '21', '85' ]
[ 'Rahul', '22', '90' ]
[ 'Vidya', '23', '95' ]
```

Reading as Dictionary (column headers as keys):

```
import csv

with open("students.csv", mode="r") as file:
```

```
reader = csv.DictReader(file)
```

```
for row in reader:
```

```
    print(row['Name'],row['Age'])
```

OUTPUT:

```
Anita 21  
Rahul 22  
Vidya 23
```

Writing as Dictionary

```
import csv
```

```
data = [
```

```
    {"Name": "Alice", "Age": 23, "Marks":70},
```

```
    {"Name": "Bob", "Age": 30, "Marks":80}
```

```
]
```

```
with open("data.csv", mode="w", newline="") as file:
```

```
    fieldnames = ["Name", "Age", "Marks"] # Added 'Marks' to fieldnames
```

```
    writer = csv.DictWriter(file, fieldnames=fieldnames)
```

```
    writer.writeheader() # writes column names
```

```
    writer.writerows(data)
```

OUTPUT:

Name,Age,Marks

Anita,21,85

Rahul,22,90

Vidya,23,95

FOR LARGE FILES(using pandas)

```
import pandas as pd
```

```
# Read CSV
```

```
df = pd.read_csv("data.csv")
```

```
print(df)
```

```
# Write CSV
```

```
df.to_csv("output.csv", index=False)
```

Illustrate one dimensional, two dimensional and multidimensional data

One-Dimensional (1D) Data

Data with only one attribute (or feature).

Representation: A list, array, or Pandas Series.

Think of: A single column of values.

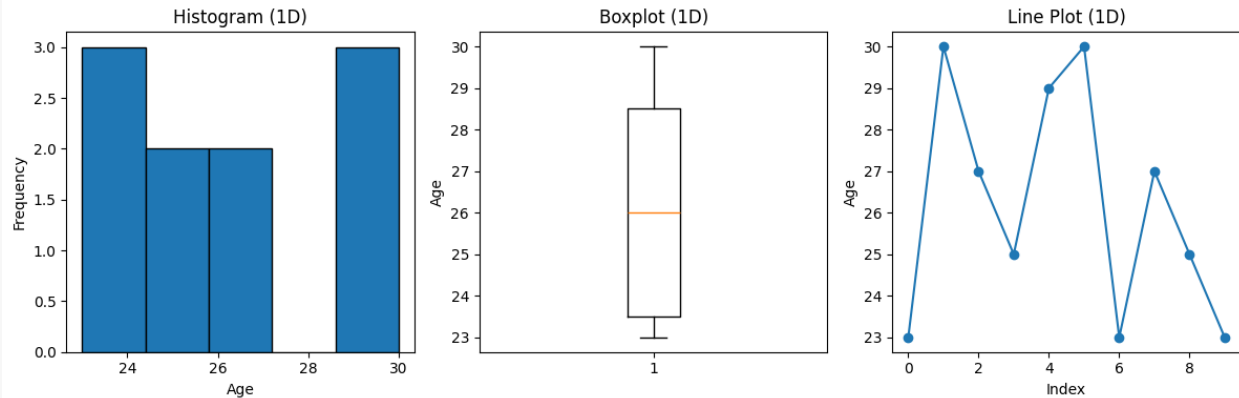
Best Visualizations:

Histogram → distribution of values.

Boxplot → summary of min, max, median, quartiles.

Line plot → if data has order (e.g., time series).

```
import matplotlib.pyplot as plt
import pandas as pd
# 1D data: list of ages
ages = pd.Series([23, 30, 27, 25, 29, 30, 23, 27, 25, 23])
plt.figure(figsize=(12,4))
# Histogram
plt.subplot(1,3,1)
plt.hist(ages, bins=5, edgecolor="black")
plt.title("Histogram (1D)")
plt.xlabel("Age")
plt.ylabel("Frequency")
# Boxplot
plt.subplot(1,3,2)
plt.boxplot(ages)
plt.title("Boxplot (1D)")
plt.ylabel("Age")
# Line plot
plt.subplot(1,3,3)
plt.plot(ages, marker="o")
plt.title("Line Plot (1D)")
plt.xlabel("Index")
plt.ylabel("Age")
plt.tight_layout()
plt.show()
```



Two-Dimensional (2D) Data

Definition: Data with rows and columns (tabular format).

Representation: A matrix, table, or Pandas DataFrame.

Think of: Excel sheet with rows (records) and columns (features).

Best Visualizations:

Multiple Histograms → each column's distribution.

Scatter Plot → relationship between two numerical variables.

Bar Chart → categorical vs numerical (e.g., Department vs Avg Salary).

Heatmap → correlations between features.

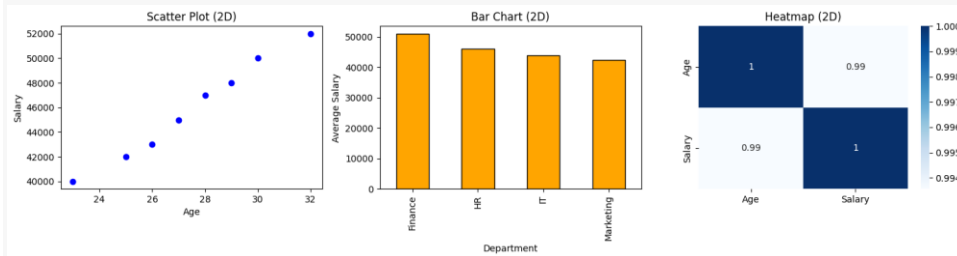
`import seaborn as sns`

`# 2D data`

```
data = {
    "Age": [23, 30, 27, 25, 29, 32, 28, 26],
    "Salary": [40000, 50000, 45000, 42000, 48000, 52000, 47000, 43000],
    "Department": ["IT", "Finance", "HR", "Marketing", "IT", "Finance", "HR", "Marketing"]
}

df = pd.DataFrame(data)
plt.figure(figsize=(15,4))
# Scatter plot (Age vs Salary)
plt.subplot(1,3,1)
plt.scatter(df["Age"], df["Salary"], color="blue")
plt.title("Scatter Plot (2D)")
plt.xlabel("Age")
plt.ylabel("Salary")
# Bar chart (Department vs Avg Salary)
plt.subplot(1,3,2)
avg_salary = df.groupby("Department")["Salary"].mean()
avg_salary.plot(kind="bar", color="orange", edgecolor="black")
plt.title("Bar Chart (2D)")
plt.ylabel("Average Salary")
# Heatmap (correlation matrix)
plt.subplot(1,3,3)
sns.heatmap(df[["Age", "Salary"]].corr(), annot=True, cmap="Blues")
plt.title("Heatmap (2D)")
```

```
plt.tight_layout()
plt.show()
```



Multi-Dimensional Data (3D and higher)

Definition: Data with more than 2 dimensions (e.g., data cube).

Representation: A 3D array, tensor, or panel (old Pandas type).

Think of: A stack of tables (2D DataFrames) arranged along a new dimension.

* List item

* List item

Best Visualizations:

Grouped Histogram / Bar Plot → distribution split by category.

3D Scatter Plot → visualize 3 variables (x, y, z).

Pair Plot (Seaborn) → all pairwise scatter plots & histograms.

Dimensionality Reduction (PCA, t-SNE, UMAP) → project to 2D/3D for visualization.

"""

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Encode Department as numbers for plotting
```

```
dept_mapping = {"IT": 0, "Finance": 1, "HR": 2, "Marketing": 3}
```

```
df["DeptCode"] = df["Department"].map(dept_mapping)
```

```
fig = plt.figure(figsize=(6,6))
```

```
ax = fig.add_subplot(111, projection="3d")
```

```
scatter = ax.scatter(df["Age"], df["Salary"], df["DeptCode"],
                    c=df["DeptCode"], cmap="viridis", s=50)
```

```
ax.set_xlabel("Age")
```

```
ax.set_ylabel("Salary")
```

```
ax.set_zlabel("Department Code")
```

```
ax.set_title("3D Scatter Plot (3D Data)")
```

```
# Legend
```

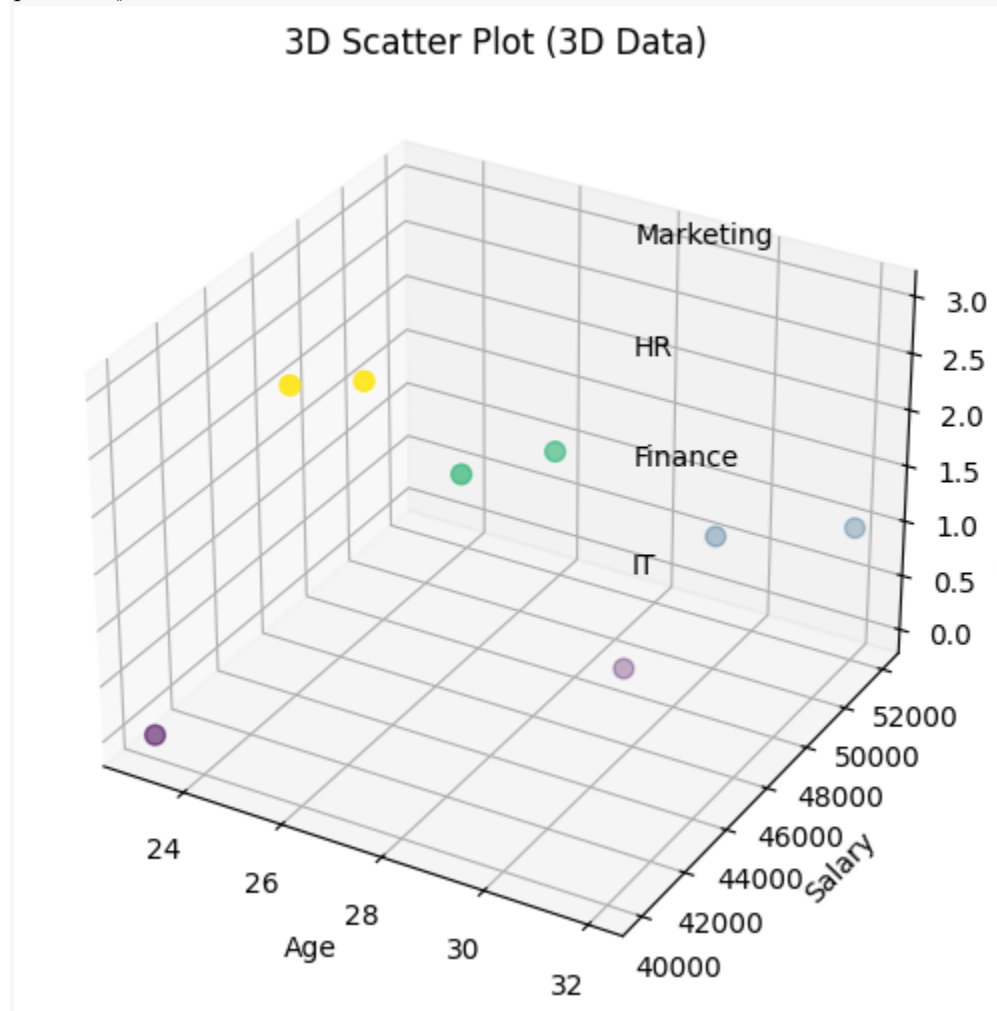
```
legend_labels = list(dept_mapping.keys())
```

```
for dept, code in dept_mapping.items():
```

```
    ax.text(df["Age"].mean(), df["Salary"].max(), code, dept)
```



```
plt.show()
```



SOLVE THE PROBLEMS ON SIMPSON PARADOX

SIMPSON PARADOX

The trend reverses when we combine the groups.

PROBLEM 1:

QUESTION

Coast	# of Members	Avg. # of Friends
West Coast	101	8.2
East Coast	103	6.5

So:

- The **West Coast** group has **101 members**, each with an **average of 8.2 friends**.
- The **East Coast** group has **103 members**, each with an **average of 6.5 friends**.

Coast	Degree	# of Members	Avg. # of Friends
West Coast	PhD	35	3.1
East Coast	PhD	70	3.2
West Coast	no PhD	66	10.9
East Coast	no PhD	33	13.4

Table 1 (Overall Averages)

- **West Coast overall:** 101 members, avg 8.2 friends
- **East Coast overall:** 103 members, avg 6.5 friends

Here it looks like **West Coast members have more friends on average**.

Table 2 (Split by PhD vs no PhD)

- **PhD subgroup:**
 - West Coast (3.1), East Coast (3.2) → East Coast PhDs have **slightly more friends**.
- **No PhD subgroup:**
 - West Coast (10.9), East Coast (13.4) → East Coast non-PhDs have **more friends**.

Within **both subgroups**, East Coast members have more friends.

The Paradox

- **Within groups (PhD vs no PhD):** East Coast > West Coast
- **Overall (all members together):** West Coast > East Coast

Simpson's Paradox is when subgroup trends are opposite to the overall trend.

Here:

- **Within each group (PhD/no PhD):** East > West
- **Overall:** West > East

UC Berkeley graduate admissions case (1973).

PROBLEM 2:

	Men		Women	
	# Applied	% Admitted	# Applied	% Admitted
Major A	825	62% admitted	108	82% admitted
Major B	560	63% admitted	25	68% admitted
Major C	325	37% admitted	593	34% admitted
Major D	417	33% admitted	375	35% admitted
Major E	191	28% admitted	393	24% admitted
Major F	373	6% admitted	341	7% admitted
Total:	2,690	44% admitted	1,835	34.5% admitted

What the table shows

- **Overall admission rates:**
 - Men: **44% admitted**
 - Women: **34.5% admitted**
 - At first glance, it looks like **men are favored** overall.
- **Within individual majors:**
 - In **every major (A–F)**, the % of women admitted is **equal to or higher** than men.
 - Within each department, **women are not disadvantaged** — in fact, sometimes they do better.

BAYES THEOREM

PROBLEMS:

PROBLEM 1:

Imagine you are a financial analyst at an investment bank. According to your research of publicly-traded companies, 60% of the companies that increased their share price by more than 5% in the last three years replaced their CEOs during the period. At the same time, only 35% of the companies that did not increase their share price by more than 5% in the same period replaced their CEOs. Knowing that the probability that the stock prices grow by more than 5% is 4%, find the probability that the shares of a company that fires its CEO will increase by more than 5%.

Question in layman language:

- Among companies whose stock price went up by more than 5%, about **60% also replaced their CEO**.
- Among companies whose stock price did **not** go up by more than 5%, about **35% still replaced their CEO**.
- Overall, only **4% of companies** saw their stock price grow by more than 5%.

Now, suppose you come across a company that **fired its CEO**.

The question is:

What are the chances that this company's stock price will actually go up by more than 5%?

Solution:

Step 1: Define events

- A: Company replaces its CEO
- B: Company's stock price grows by more than 5%

Step 2: Given values

$$P(A | B) = 0.60$$

$$P(A | \text{not } B) = 0.35$$

$$P(B) = 0.04, \quad P(\text{not } B) = 0.96$$

,

$$P(A) = P(A | B)P(B) + P(A | \text{not } B)P(\text{not } B)$$

$$P(B|A) = \frac{P(A|B) \cdot P(B)}{P(A|B) \cdot P(B) + P(A|\neg B) \cdot P(\neg B)}$$

Step 3: Numerator

$$P(A | B) \cdot P(B) = 0.60 \times 0.04 = 0.024$$

Step 4: Denominator

$$\begin{aligned}P(A) &= (0.60 \times 0.04) + (0.35 \times 0.96) \\&= 0.024 + (0.35 \times 0.96) \\&= 0.024 + 0.336 \\&= 0.36\end{aligned}$$

Step 5: Apply Bayes' theorem

$$\begin{aligned}P(B | A) &= \frac{P(A | B) \cdot P(B)}{P(A)} \\P(B | A) &= \frac{0.024}{0.36} \\P(B | A) &= 0.0667\end{aligned}$$

Ans: If a company replaces its CEO, the probability that its stock price will rise by more than 5% is about **6.7%**.

PROBLEM 2:

In a certain city, 20% of the population has a rare genetic condition. There is a diagnostic test available for this condition that correctly identifies 95% of those who have the condition and incorrectly identifies 10% of those who do not have it as positive. If a randomly selected individual takes the test and the result is positive, what is the probability that the individual actually has the rare genetic condition?

Question in layman language:

In a city, 20 out of every 100 people have a rare genetic disease.
There is a medical test for this disease:

- If a person **has the disease**, the test shows **positive 95% of the time**.
- If a person **does not have the disease**, the test still shows **positive 10% of the time** (false alarm).

Now, if a random person takes the test and the result is **positive**, what's the chance that the person **actually has the disease**?

Where:

- A: Individual has the rare condition.
- notA: Individual does **not** have the condition.
- B: Test result is positive.

Given:

- $P(A)=0.20$
- $P(\text{not } A)=0.80$
- $P(B | A) = 0.95, \quad P(B | \text{not } A) = 0.10$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B | A)P(A) + P(B | \neg A)P(\neg A)}.$$

Step 1: Compute numerator

$$P(B | A) \cdot P(A) = 0.95 \times 0.20 = 0.19$$

Step 2: Compute denominator

$$\begin{aligned} P(B) &= P(B | A)P(A) + P(B | \text{not } A)P(\text{not } A) \\ &= (0.95 \times 0.20) + (0.10 \times 0.80) \\ &= 0.19 + 0.08 = 0.27 \end{aligned}$$

Step 3: Apply Bayes' theorem

$$\begin{aligned} P(A | B) &= \frac{P(B | A) \cdot P(A)}{P(B)} \\ P(A | B) &= \frac{0.19}{0.27} \approx 0.7037 \end{aligned}$$